

THIRANEX INTERNSHIP | CYBER SECURITY | SECURE LOGIN SYSTEM

INTERNSHIP PROJECT REPORT

Thiranex – Skill Development & Future Tech
Secure Login System
Cyber Security-Based Web Authentication Tool

Intern Details

Field	Information
Intern Name	V Subhaharini
Intern ID	THX-MAY2626-926
Role	Intern – Cyber Security
Organization	Thiranex
Duration	May 2026 – June 2026
Work Mode	Remote / Project-Based
Date of Submission	12 June 2026
Project	Secure Login System

1. Introduction

This report presents the Secure Login System developed as part of the Cyber Security internship at Thiranex. The project focuses on building a fully functional web-based authentication system that enables secure user registration and login using hashed passwords, session management, and SQL injection protection, built with Python Flask and SQLite.

Credential theft and unauthorized access are among the most critical cyber threats. A well-implemented login system with bcrypt password hashing, parameterized queries, and session handling is a foundational defense against such attacks. This project demonstrates these security principles in a working web application.

2. Objectives

- To build a Python Flask-based secure login web application with user registration and authentication.
- To implement bcrypt password hashing (using the Werkzeug library) to protect stored credentials.
- To integrate SQLite as the backend database with parameterized queries to prevent SQL injection.
- To implement session management with secure login/logout functionality.

- To create a clean, functional front-end using HTML templates (Jinja2 via Flask).
- To verify data integrity by querying the database and confirming hashed password storage.

3. Tools & Technologies Used

Tool / Technology	Purpose
Python 3.x	Core programming language for building the backend logic.
Flask	Lightweight web framework for routing, sessions, and template rendering.
SQLite3	Embedded relational database for storing user credentials.
Werkzeug (bcrypt)	Secure password hashing using generate_password_hash / check_password_hash.
Jinja2 (HTML Templates)	HTML templating engine for login, register, and home pages.
VS Code	Code editor used to write and execute the project.
PowerShell Terminal	Used to run the Flask app and test endpoints.

4. Methodology

The Secure Login System follows a standard web authentication pipeline. User input is collected through HTML forms, validated server-side, and stored securely with bcrypt-hashed passwords in a SQLite database. Sessions are used to maintain authenticated state across requests.

4.1 Project Structure

- **app.py** – Main Flask application with all routes and business logic.
- **templates/login.html** – Login form page.
- **templates/register.html** – User registration page.
- **templates/home.html** – Protected home page shown after successful login.
- **users.db** – SQLite database storing user credentials.

4.2 Application Pipeline

Step	Component	Description
1	Database Setup	SQLite table created with id, username, and hashed password fields.
2	User Registration	Username and password collected; password hashed with bcrypt before storage.
3	SQL Injection Protection	Parameterized queries (?, ?) used for all DB operations.
4	User Login	Stored hash verified using check_password_hash; session created on success.
5	Session Management	Flask session stores authenticated username; logout clears session.
6	Protected Route	/home route checks session before rendering welcome page.

5. Code Overview

The project was implemented in Python in a single main file: app.py. Below is a summary of the key code sections:

- **Import & Setup:** Imports Flask, session, redirect, url_for, request, render_template from flask; sqlite3 for database; werkzeug.security for generate_password_hash and check_password_hash. A secret key is set for session encryption.
- **Database Initialization:** A function init_db() creates the users table (if not exists) with columns: id (INTEGER PRIMARY KEY), username (TEXT UNIQUE), password (TEXT). It is called on app startup.
- **Registration Route (/register_user – POST):** Collects username and password from form. Hashes the password using generate_password_hash(). Inserts the username and hashed password into the database using parameterized query: INSERT INTO users (username, password) VALUES (?, ?).
- **Login Route (/login – POST):** Fetches user record from DB using parameterized SELECT WHERE username=?. Calls check_password_hash() to verify submitted password against stored hash. If valid, stores username in session and redirects to /home.
- **Home & Logout Routes:** The /home route checks if the session contains a logged-in username. If yes, renders home.html with a welcome message. The /logout route calls session.clear() and redirects to the login page.

6. Actual Output (Live Run)

The following output was captured from an actual run of the program in VS Code terminal on 12 June 2026:

```
PS C:\secure login system> python app.py
```

```
* Serving Flask app 'app'
```

```
* Debug mode: on
```

```
WARNING: This is a development server. Do not use it in a production deployment.
```

```
Use a production WSGI server instead.
```

```
* Running on http://127.0.0.1:5000
```

```
Press CTRL+C to quit
```

7. Project Screenshots

Screenshot 1: VS Code Terminal & SQLite Verification

The database integrity was verified by running direct queries inside the interactive terminal. Executing `SELECT * FROM users` shows that for the user **subhaharini**, the password is encrypted as a byte-string starting with `b'$2b$12$...` instead of clear plain text, proving that password hashing works perfectly.

Screenshot 2: Login Page Interface

The web interface running locally at `http://127.0.0.1:5000/`. It provides a clean structure containing **Username** and **Password** text boxes, a **Login** button, and a redirecting hyperlink labeled *New User? Register Here*.

Screenshot 3: Secured Home Dashboard

After entering valid credentials, the application routes the user to the protected `/home` URL path. It renders a customized message stating **"Welcome subhaharini - Login Successful!"** along with a clear **Logout** option.

Screenshot 4: Flask Development Server Logs

The terminal captures operational logs recording step-by-step HTTP traffic requests. It displays successful resource loading events: `GET /login`, `POST /register_user`, `POST /login` yielding a 302 redirection flag, and `GET /home` executing with a healthy 200 OK status.

8. Learning Outcomes

- Understood how bcrypt password hashing works and why plain-text password storage is a critical security vulnerability.
- Learned how to implement parameterized SQL queries in Python to prevent SQL injection attacks.
- Gained hands-on experience with Flask session management for maintaining authenticated user state.
- Developed an understanding of common web authentication attack vectors including brute force, credential stuffing, and injection attacks.
- Learned to structure a Flask web application with separate routes, HTML templates, and a database layer.
- Practiced building an end-to-end secure web project with clean, modular Python code.

9. Challenges Faced & Solutions

Challenge	Solution
Ensuring passwords are never stored in plain text	Used Werkzeug's <code>generate_password_hash()</code> which applies bcrypt automatically.
Preventing SQL injection in login and registration	Used parameterized queries with <code>?</code> placeholders instead of string formatting.
Managing user sessions securely across requests	Set a strong Flask secret key and used <code>session.clear()</code> on logout.
Handling duplicate username registration	Added UNIQUE constraint on username column; handled <code>IntegrityError</code> in Python.

10. Future Improvements

- Implement Two-Factor Authentication (2FA) using TOTP (e.g., Google Authenticator) for an additional security layer.
- Add input validation with minimum password length, special character requirements, and username sanitization.
- Integrate rate limiting and account lockout after repeated failed login attempts to prevent brute force attacks.
- Migrate to PostgreSQL for production-grade database reliability and scalability.
- Add HTTPS/TLS support and deploy on a cloud platform (e.g., Heroku, Render) using a production WSGI server like Gunicorn.
- Implement password reset via email with secure time-limited tokens.

11. Conclusion

The Secure Login System was successfully developed and executed as part of the Thiranex Cyber Security Internship. The Python Flask application effectively implements user registration and login with bcrypt-hashed passwords, parameterized SQL queries, and session-based authentication. The system correctly stored hashed passwords in the SQLite database (confirmed via direct DB query showing `$2b$12$... bcrypt` format) and successfully authenticated users, as demonstrated by the landing home page. Flask server logs confirmed the correct HTTP request flow including POST `/login` (302 redirect) and GET `/home` (200). This project provided strong foundational skills in web security, authentication design, and practical Python web development.

12. Declaration

I hereby declare that this project and report have been prepared by me independently as part of my internship at Thiranex. The code was written and executed by me personally, as evidenced by the screenshots attached above.

Name: V Subhaharini

Intern ID: THX-MAY2626-926

Date: 12 June 2026

Organization: Thiranex – Skill Development & Future Tech